Computer Communications 248 (2026) 108414

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Context-aware anomaly detection by community detection in the Internet of Things

Fatemeh Stodt [a,b], Christoph Reich [a], Fabrice Theoleyre [b,*]

[a] Institute for Data Science, Cloud Computing and IT Security (IDACUS), University of Applied Sciences Furtwangen, 78120 Furtwangen, Germany
[b] ICube Laboratory, CNRS/University of Strasbourg, Pole API, Boulevard Sebastien Brant, 67412 Illkirch Cedex, France

ARTICLE INFO

Keywords:
Network anomaly detection
Community detection
IoT
Cybersecurity
Context-awareness

ABSTRACT

This paper introduces a novel context-aware anomaly detection framework for the Internet of Things, leveraging community detection in multi-edge graphs with a heterogeneous Graph Neural Network (HeteroGNN) architecture to enhance network security. The proposed framework detects anomalies such as unexpected communication patterns among devices that rarely interact, unusual traffic spikes during off-hours, or deviations in the contextual and knowledge-based interactions of devices. For example, in an industrial IoT environment, unauthorized access or malicious activity can be inferred from unexpected communication within a device community after working hours. Our detection approach uses multi-edge graphs to model diverse interactions (network communication, context, knowledge) and applies community detection to capture stable graph structures. By incorporating these insights into a HeteroGNN, the framework effectively distinguishes anomalous edges while maintaining scalability and adaptability to dynamic network conditions. Experimental evaluation on the CIC-ToN-IoT and CIC-IDS2017 dataset demonstrates the framework's superior accuracy, precision, and robustness, establishing it as a practical and effective solution for securing IoT networks against both known and emerging threats.

1. Introduction

The integration of Internet of Things (IoT) devices into our daily lives has been transformative, offering unprecedented convenience and efficiency in various domains such as healthcare, home automation, and smart meters. They have also started to be deployed in more constrained environments for critical applications, leading to the so-called IIoT. IIoT is a key enabler for Industry X.0: measurements are collected in real-time to feed a controller that pushes commands to actuators.

Because of its distributed nature, IoT must face a wide spectrum of threats. Attacks can come from outside (e.g., malware injection, network scans) and from inside (e.g., devices physically compromised after a local firmware update). In these conditions, we must preserve the integrity of the network infrastructure and each device. The complexity and dynamic nature of IoT networks require advanced anomaly detection systems that are context-aware, capable of discerning the subtle nuances of legitimate network behavior from malicious activities [1]. Traditional approaches [2] often fail to address the dynamic and multi-faceted nature of modern networks, especially those incorporating IIoT devices.

Existing anomaly detection methods in IoT networks primarily focus on analyzing network communication without considering the broader context in which these communications occur. In this regard, context refers to the situational factors surrounding network events, including but not limited to the time of communication, the specific devices involved, operational conditions, and expected behavior patterns. For instance, an IIoT environment may experience unauthorized communications among machines outside normal working hours, indicating potential data exfiltration or compromised devices. Similarly, traffic between unrelated devices, such as a thermostat and a security camera, could signal lateral malware propagation. This gap highlights the need for context-aware solutions that integrate such factors to provide a more robust and accurate anomaly detection mechanism.

Anomaly detections are vital for safeguarding networks against unauthorized access and attacks, employing statistical analysis, deep learning (DL), and various classification methods to enhance detection capabilities [3]. Statistical techniques identify deviations in network behavior, while DL models, particularly Convolutional Neural Networks (CNNs) and Recurrent Neural Networ (RNN), excel in learning complex patterns from data. Classification techniques, ranging from supervised learning (e.g., Decision Trees, SVM) to unsupervised and

* Corresponding author.
E-mail addresses: fatemeh.stodt@hs-furtwangen.de (F. Stodt), christoph.reich@hs-furtwangen.de (C. Reich), fabrice.theoleyre@cnrs.fr (F. Theoleyre).

https://doi.org/10.1016/j.comcom.2026.108414
Received 28 January 2025; Received in revised form 25 July 2025; Accepted 5 January 2026
Available online 6 January 2026

semi-supervised learning, are crucial in distinguishing between normal and malicious activities. Popular datasets are highly imbalanced [4] which complicates the detection and identification of attacks.

Context-Aware Anomaly Detection offers a promising technique to overcoming the limitations of traditional methods. Context-aware systems integrate contextual information such as *time*, *location*, *device state*, and *interaction history* into the anomaly detection process. This integration enables the detection of complex, context-aware anomalies that traditional methods might overlook, such as unusual communication patterns tied to specific times or locations [5].

The graph structure effectively captures the complex, dynamic, and large-scale nature of data generated by IoT networks [6].

For example, Graph Neural Networks (GNNs) can capture temporal and structural patterns within communication graphs to detect unexpected communities of devices forming at unusual times, which may indicate coordinated attacks or network compromise [7].

Our work introduces a novel anomaly detection model leveraging a multi-edge graph and community detection to gain deeper insights into network behaviors and device interrelationships. Unlike existing approaches, which often rely on predefined patterns or physical deviations, our model delves into the context of anomalies, factoring in timing and intricate interaction patterns within network traffic.

The contributions of this paper are threefold:

1. **Integration of Multi-Edge Graphs for Context-Awareness**: Unlike traditional approaches that focus solely on communication data, our method incorporates additional contextual information, such as temporal patterns, device states, and operational conditions. This multi-dimensional representation enables a more nuanced understanding of network behaviors and facilitates the detection of complex anomalies that traditional methods might miss, such as coordinated attacks or anomalous communication during off-peak hours.
2. **Use of GNN**: we exploit GNN, with a community-based approach. By combining community detection with GNN-based learning, we efficiently capture the structural and temporal dynamics of IoT networks. This allows us to detect anomalies not just at the node or edge level but across entire communities of devices, enhancing detection accuracy and scalability.
3. **Real-Time Framework**: Our framework employs a semi-supervised learning approach, utilizing a HeteroGNN to classify network edges as benign or anomalous. By leveraging multi-edge graphs and community detection through computationally efficient algorithms, such as Label Propagation Algorithm (LPA), our method operates in real-time. This design ensures applicability in resource-constrained IoT environments while providing robust anomaly detection.

We implemented our context-aware anomaly detection scheme, and make our implementation freely available [8] for the sake of reproducibility. Our method exhibits an AUC of 99% on the CIC-ToN-IoT dataset.

The paper is organized as follows: Section 2, reviews related work in the field. Section 3 explains the problem we tackle in this article. Section 4, details how we propose to detect anomalies. Section 5, presents our experimental setup, datasets, and evaluation metrics. Finally, Section 6 insights gained from our analysis and concludes the paper.

2. State of the art

This section reviews key approaches in anomaly detection for IoT networks. We first examine the anomaly detection methods and most popular Intrusion Detection Systems (IDS) datasets. Next, we explore Anomaly Detection with Graph-Based methods that utilize graph structures to identify irregularities and GNN, focusing on how deep learning enhances detection by capturing complex relationships. We then discuss the role of Context-awareness in improving detection accuracy. Finally, a Summary synthesizes the key insights and identifies research gaps.

2.1. Statistical-based anomaly detection

Statistics-based techniques aim to statistically model normal network behavior, i.e, behavior not influenced by attacks. When collected metrics deviate significantly from the established statistical model, the anomaly detector triggers an alarm. Gaussian models, histograms, and z-scores are commonly employed to define this statistical baseline [9]. However, these methods face substantial challenges in dynamic environments, such as rapidly fluctuating network traffic patterns. Additionally, they often struggle to capture the complex, non-linear relationships that characterize modern data streams, limiting their effectiveness in detecting sophisticated anomalies.

2.2. Machine learning-based anomaly detection

This method learns patterns directly from data and requires labeled datasets to train models that classify normal and anomalous behavior effectively. For instance, the Support Vector Machine technique can be used for classification [10]. It maps data into a high-dimensional space and identifies a separating hyperplane that distinguishes normal data from potential anomalies. Fosic et al. [11] have assessed the detection accuracy of various algorithms (Stochastic Gradient Descent (SGD), Support Vector Machines (SVM), K-Nearest Neighbor (K-NN), Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), AdaBoost (AB)) with the UNSW-NB15 dataset. However, these methods necessitate labeled datasets and fail to identify zero-day attacks, which are previously unidentified security vulnerabilities that attackers exploit before they receive a patch or identification. In this context, Graph2vec+RF [12] introduces a unique approach by constructing flow graphs from initial bidirectional network packets. The system embeds these graphs and classifies them with a Random Forest model. This lightweight method excels in early detection with minimal labeled data but may struggle to adapt to evolving attack patterns.

2.3. Deep learning-based anomaly detection

To detect sophisticated attacks, Deep Learning techniques may be particularly useful. In particular, Variational Autoencoders (VAE) can detect anomalies [13]. VAEs project input data into a lower-dimensional space. By subsequently minimizing the reconstruction error, VAEs allow for selecting the most representative projection of the original data. A significant reconstruction error is therefore assumed to correspond to abnormal traffic. This method is particularly attractive since it does not require labeled data: the VAE is trained uniquely with normal traffic, and can thus detect zero-day attacks.

Min et al. [14] reduce the memory consumption with a memory-augmented deep auto-encoder (MemAE). They implement memory networks to store and recall normal data patterns, improving the accuracy of detecting deviations in network traffic.

Packet transmissions can be represented as time series, with anomaly detection focusing on identifying deviations within these series. Thus, Ullah et al. [15] rely on Recurrent Neural Network (RNN) to model flows. They compare Long Short Term Memory (LSTM), BiLSTM, and Gated Recurrent Unit (GRU) techniques to construct this RNN. Trinh et al. [16] train a stacked Long Short-Term Memory (LSTM) to model traffic in a cellular network. The system can for instance detect a rapid growth of the number of users, representing a Denial of Service attack. However, the approach focuses uniquely on radio access, with a reduced data dimensionality. Elsayed et al. [17] propose to use first a Long Short Term Memory (LSTM) autoencoder to construct a temporal representation of the time-series. Then, a one-class SVM detects the boundaries of the *normal* data.

Hybrid approaches combine multiple detection methods, leveraging their strengths. Tang et al. [18] combine a stacked autoencoder with One-Class SVM: the autoencoder reduces the data dimensionality before classification. Sahu et al. [19] use first a Convolutional Neural Network

(CNN) to extract features of interest locally, and then use a collection of LSTM to classify the different behaviors. However, combining different techniques increases the complexity and requires more computational resources.

The complexity of Deep Learning techniques remain high. Thus, to reduce the data dimensionality, 3D-IDS [20] integrates feature disentanglement with dynamic graph diffusion to separate attack-specific features from non-attack-specific ones. LUCID (Lightweight Deep Learning DDoS Detection) [21] reduces the number of layers and parameters to reduce the computational complexity of a CNN.

2.4. Anomaly detection with GNN

Graphs are widely used to model communication networks [22]. Thus, it seems natural to use the recently proposed GNN [23] technique, that adapted Deep Learning to graph structures.

GNNs rely on a message-passing approach: the node embeddings are passed from one node to another through the edges. The application of GNNs in anomaly detection involves analyzing network transactions, social networks, or any system that can be represented as a graph to identify irregular and potentially malicious activities. GNN can also be applied inversely to generate attacks more complex to detect [24]. GNNs have been employed for anomaly detection in IIoT systems across a range of applications, including smart transportation, smart energy, and smart factories [25]. Sec2graph [26] tries to detect novelty when constructing a graph structure of events from log files.

E-GraphSAGE [27] made a pioneering piece of work to adapt GNN for the detection of IoT attacks. While classical GNN are used for node's classification, E-GraphSAGE detects anomalous flows (edges). Thus, the authors modify the aggregation function of the GNN: the node embedding is computed with the features of all the *edges* with neighbors. By repeating the aggregation, E-GraphSAGE can capture the dependencies farther away in the network. E-GRACL [28] extends the GNN approach by improving the sampling strategy to enhance the feature representation.

Altaf et al. [29] extend this work to deal with multigraph. Multiple edges may for instance exist if multiple flows connect two IoT nodes. The authors introduce a spatial layer with an attention function between two Graph Convolution Networks for traffic classification. Friji et al. [30] adopt a different approach, where a node represents a flow. An edge exists between two flows if they share the same source or destination. The weight of an edge is the similarity between the two corresponding flows. Thus, the GNN can be directly applied to the flows, using the graph structure. An attention-based feature extractor in parallel with a spatial feature extractor (i.e, a GNN) identifies anomalies. Park et al. [31] integrate multiple graph types and refine embeddings through attention mechanisms.

Capturing the dynamics in the graph may be relevant to detect anomalies. AnomRank [32] monitors abrupt changes in node importance scores by counting the number of edge insertions or deletions associated with the node's significance in a social network.

Temporal Graph Networks (TGNs) [33] manage evolving graph structures, applied to social networks. The node embedding evolves, and an aggregator computes average embedding to reduce the complexity. EULER [34] combines GNN and RNN in a massively parallel architecture to make the detection faster. However, it still requires a huge amount of computational resources.

However, communication networks exhibit different characteristics compared to social networks (e.g., communication patterns tend to be more repetitive).

2.5. Context-aware anomaly detection

Context-awareness is the capacity to improve actions or decision-making within a particular context by leveraging pertinent environmental and situational information, such as time, location, and user activity. It helps distinguish unusual but safe activities from real dangers. SSDCM [35] learns the representation of a multi-layer network, i.e, each layer representing a different context. This technique relies on cluster and node embeddings to derive clusters in complex graph topologies, aggregating the different contexts.

We can also consider uncertainability in context awareness [36]. We should assess the *quality* of the context. Contextual belief correction (CBC) helps to capture such uncertainty at the cost of a more complex model. Rullo et al. [37] considers also the risk and exploits a taxonomy of attacks according to the manufacturer However, the framework remains conceptual and does not detect zero-day attacks.

Additional information can also be exploited directly at the application level to identify contextual anomalies. For instance, IoT-CAD [38] clusters devices with similar sensor readings to generate application-specific fingerprints and then uses a Long Short-Term Memory (LSTM) neural network combined with a Gaussian estimator to detect unexpected behaviors. In automotive domains, sensor semantics can highlight suspicious activity for example, an unexpected door-opening event while the vehicle is in motion [39]. Similarly, DyEdgeGAT [40] constructs dynamic graphs of sensor signals and applies graph attention mechanisms to identify anomalous interactions over time.

Although these application-aware techniques leverage rich contextual information, they remain highly tailored to specific use cases and require implementing bespoke models at the application level. For instance, IoT-CAD [38] depends on training LSTMs per device type using sensor values to model expected behavior, and DyEdgeGAT [40] constructs dynamic graphs with attention mechanisms to detect anomalous interactions. These approaches are very complementary to ours: while we focus on the communications to detect intruders and attacks, other approaches can use the measurements themself to detect misbehaving sensor (at the application level). Our proposed multi-edge, community-based graph approach integrates network, contextual, and knowledge features directly at the communication layer.

Table 1
Comparison of different anomaly detection approaches.

Paper	IoT network	Context-aware	Graph-based	Real-time
[27]	✓	✗	✓	✓
[30]	✓	✗	✓	✓
[40]	✓	✓	✓	✗
[38]	✓	✓	✗	✓
[36]	✗	✓	✗	✗
[20]	✓	✗	✓	✓
Ours	✓	✓	✓	✓

2.6. Summary

Table 1 summarizes the most important references about anomaly detection. The table highlights key features across various approaches, including IoT network applicability, context-awareness, graph-based methodologies, self-supervised learning, and real-time capabilities.

Many existing works exploit graph-based approaches. However, they lack context-awareness capabilities, which are crucial for dynamic and evolving IoT environments. Zhao et al. [40] and Yasaei et al. [38] incorporate context-awareness, but they have no real-time capabilities.

Despite significant progress in anomaly detection for IoT networks, a major gap remains: no existing method comprehensively integrates all the critical features context-awareness, graph-based modeling, self-supervised learning, and real-time capability necessary for addressing the dynamic and resource-constrained nature of IoT environments. Additionally, many approaches either focus on specific applications or struggle to scale effectively across diverse IoT scenarios.

Our work directly addresses this gap by ticking all the boxes in Table 1. Specifically, we present a unified framework that combines multi-edge graph representations, community detection, and GNN to

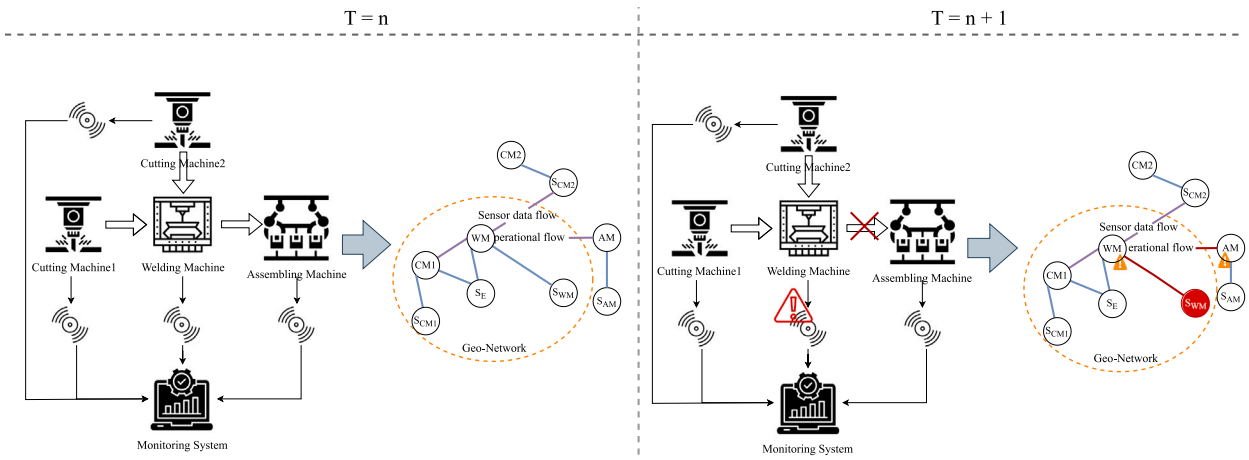


Fig. 1. Attack scenario.

create a robust anomaly detection system. Unlike previous approaches, our method captures the temporal and contextual intricacies of IoT traffic while maintaining the computational efficiency needed for real-time deployment. Achieving this integration is non-trivial due to the inherent challenges of IoT networks, such as heterogeneous device interactions, rapid data changes, and the need for low-latency responses. In the next section, we detail how our methodology overcomes these challenges and delivers a comprehensive solution for IoT anomaly detection.

3. Problem statement

IoT faces many challenges in terms of security. As the environment is inherently distributed, threats can come from both inside and outside. However, securing the infrastructure must not prevent the multitude of tasks (e.g., maintenance, predictive maintenance, reporting).

In this context, our approach is designed to detect two types of *anomalies*:

Attacks: Malicious persons try to break into the system to gather information, or even to jeopardize the safety of Cyber Physical Systems. For instance, a machine could be attacked so that it operates outside its safety zone in a connected smart factory [41].

Misbehavior: the system does not behave as expected. When considering the network infrastructure, it may consist in

- *Temporal Anomalies*: Communication occurring at unexpected times (e.g., after work hours).
- *Behavioral Anomalies*: New or unusual interactions between devices.
- *Statistical Anomalies*: Deviations in metrics like packet size distribution or flow duration.

In IIoT environments, machines are expected to form tightly connected communities based on their typical interactions. For instance, a packaging machine usually communicates with its conveyor belt and quality control system, forming a stable community over time. However, if this packaging machine suddenly starts communicating with a financial server or an unrelated printer, the community structure is disrupted, signaling an anomalous interaction. By using GNNs, we can detect such irregularities in the communication graph by identifying nodes or edges that deviate from the expected community patterns. This approach enables the system to flag suspicious activities like malware spreading or unauthorized access.

3.1. IIoT scenario and targeted anomalies

In this section, we present a simplified and generic representation of an IIoT network to illustrate the types of attacks and anomalies that can be detected using our context-aware anomaly detection algorithm. The focus is on demonstrating how contextual information, such as environmental conditions and expected node behaviors, enhances the system's ability to identify security threats.

3.1.1. Generic IIoT network

A typical IIoT network consists of interconnected devices, including machines, sensors, gateways, and a centralized monitoring system (see Fig. 1). These devices communicate to support industrial operations, such as production workflows and system monitoring. The network operates under predefined policies, specifying expected communication patterns, authorized devices, and acceptable operating times.

Under normal conditions, sensor data flows from devices to the Monitoring System, and machines exchange operational commands within the constraints of defined workflows. For example, a machine might communicate only with its associated sensors or the Monitoring System during production hours.

To demonstrate the detection capabilities of the algorithm, we highlight two generic anomaly scenarios:

3.1.2. Scenario 1: Unauthorized device access

An attacker introduces a rogue device into the network to impersonate a legitimate node. This device attempts unauthorized actions, such as communicating with multiple machines or accessing the Monitoring System outside of allowed hours. Examples of anomalous behaviors include:

- Establishing unexpected communication patterns, such as sending commands to machines it is not authorized to control.
- Attempting to access the network during non-production hours, violating temporal access policies.

These behaviors disrupt the contextual integrity of the network and can be detected by identifying deviations in communication patterns and access times.

3.1.3. Scenario 2: Misconfigured or compromised device behavior

A legitimate device in the network, such as a machine or sensor, becomes compromised or misconfigured, leading to abnormal behavior. This could involve:

- Communicating with devices it typically does not interact with, indicating a structural anomaly in the network.

- Sending an unusually high volume of data or operating during non-production hours, suggesting temporal and statistical anomalies.

Such deviations indicate a violation of contextual norms, flagging potential security breaches or misconfigurations.

3.2. Network-related information

We would like to detect anomalies by just inspecting the network traffic. Thus, we capture all the packets that we collect in a centralized entity, executing our anomaly detection scheme.

We need for instance to exploit:

- **Temporal Features**: We incorporate both *active/idle times* and summarized *packet inter-arrival times* to comprehensively detect anomalies at multiple granularities. Active and idle times succinctly summarize the overall behavior at the flow level, capturing macroscopic anomalies such as unexpected long periods of inactivity or unusual activity patterns. Conversely, inter-arrival times provide a more granular view suitable for identifying subtle packet-level anomalies (e.g., covert channels, stealthy scans) that broader flow-level summaries might miss. To balance accuracy and computational efficiency, rather than using raw inter-arrival series directly, we summarize these series using statistical descriptors (mean, median, standard deviation, and critical percentiles such as the 90th and 99th percentiles). This approach significantly reduces dimensionality and ensures computational efficiency while retaining the capacity to detect fine-grained anomalies.
- **Traffic Metrics** (e.g., packet counts, flow duration) to identify high-volume attacks like DDoS.
- **Flow Flags and Protocol Features** to detect unauthorized use or misuse of protocols, such as SYN scans.
- **Inter-Arrival Times and Bandwidth Usage** to represent hidden data exfiltration or lateral movement.

We need labeled datasets, that we will use for training. By learning usual properties, we can detect deviations, and thus anomalies.

3.3. Community-based approach

We propose here a community detection solution based on a multi-edge graph. Industrial systems are often organized with a sensor/controller/actuator workflow. In other words, the traffic pattern is restricted, and a limited set of devices need to communicate. Therefore, our method is based on detecting *communities*, i.e, clusters of devices that communicate with one another, and observing their evolution over time for anomaly detection.

Context is the information and attributes that describe the environment, situation, and timing of a system, influencing its behavior. It helps understand patterns, detect deviations, and distinguish between normal and anomalous activities. By integrating contextual information into our multi-graph, we enhance the ability to accurately identify anomalies and ensure robust monitoring of industrial systems.

We consider the following types of interactions between two IP endpoints:

- **Network Communication Features**: Metrics directly related to the flow of packets, connection protocols, and header-level attributes such as:
  - *Source and Destination IPs/Ports*: Identify the IP endpoints of communication flows.
  - *Protocol Type*: Specifies the protocol used in the communication (e.g., TCP, UDP, ICMP).
  - *Flow Duration*: Measures the time elapsed between the first and last packet of a flow.
  - *Packet and Byte Counts*: Total number of packets and bytes exchanged in the flow.
- **Contextual Features**: Attributes capturing temporal, behavioral, or contextual properties of the traffic (e.g., timing, interaction patterns). These features focus on the interaction dynamics and external factors influencing behavior over time:
  - *Timestamp*: Records the time of each flow, allowing temporal pattern analysis.
  - *Idle and Active Times*: Periods of inactivity and activity between consecutive packets in a flow which we use "Mean Idle Time", and "Mean Active Time" for it.
  - *Inter-Arrival Times*: Time intervals between successive packets in a flow.
- **Knowledge-Based Features**: Knowledge Features encode policies, rules, and expected norms governing the network's operation. These features define the logical relationships and constraints among devices, including:
  - *Packet Size Averages*: Average sizes for packets, forward segments, and backward segments (*Pkt Size Avg*, *Fwd Seg Size Avg*, *Bwd Seg Size Avg*).
  - *Flow Flags*: Protocol-specific flags (e.g., SYN, ACK) indicating the state or purpose of a packet.

These features are collected at both the packet and the flow levels. Flow-level aggregation provides a higher-level view of communication patterns. Features like inter-arrival times and flow flags are particularly useful for detecting anomalies related to malicious behavior, such as unusual traffic bursts or protocol misuse.

4. Context-aware community-based multi-graph anomaly detection

This section introduces our pipeline for anomaly detection, integrating multi-edge graph construction, community detection, and a HeteroGNN [42].

4.1. Big picture and steps orchestration

The pipeline, illustrated in Fig. 2, combines temporal, contextual, and structural data to detect anomalies in dynamic network environments. Algorithm 1 provides an overarching view of how these components interact.

The pipeline begins with network traffic data collection, followed by preprocessing to retain relevant features. The preprocessed data is used to construct a time-evolving multi-edge graph $G_t = (V_t, E_t)$, where V_t represents the nodes (IP entities) and E_t denotes edges capturing multiple interaction types. Each edge type (communication, context, knowledge) is annotated with weights computed from normalized features, ensuring a detailed representation of network interactions. Communities are then detected within G_t to identify clusters of nodes with similar patterns, providing contextual insights into network behavior.

The enriched graph, with annotated nodes and edges, is processed using a HeteroGNN for edge classification, flagging anomalous edges. The pipeline operates in a sliding window framework $[t-W, t]$, ensuring real-time updates as new data arrives.

4.2. Time-dependent multi-graph construction

To detect communities, we construct a graph representation of the network and the different interactions between the different IP entities. More precisely, we rely on the multigraph structure since we need to consider multiple edges between the same pair of vertices, corresponding to different types of interaction.

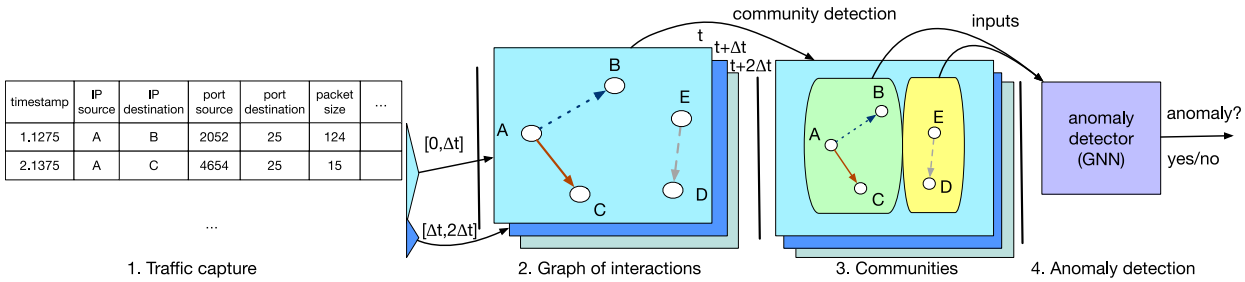


Fig. 2. Steps of our community-based anomaly detection approach.

Algorithm 1 Sliding Window Orchestration

Require: W: window size
Require: Δt: frequency of updates
Require: HeteroGNN model (trained or partially trained)
Require: θ: anomaly threshold
Initialize $\mathcal{B}_0 \leftarrow \emptyset$ ▷ initial flow buffer
$t \leftarrow 0$
while True **do**
  Wait until $t + \Delta t$
  $t \leftarrow t + \Delta t$
  Update buffer: Remove flows older than $t - W$, add new arrivals Δ_t
  Build $G_t(V_t, E_t)$ using Algorithm 2
  Run label propagation to compute community labels π_t
  for all $v \in V_t$ **do**
    $X_t(v)$.community_label $\leftarrow \pi_t(v)$
  end for
  Convert (V_t, E_t, X_t) to `HeteroData` structure
  Perform inference with HeteroGNN: $Z_t \leftarrow \text{HeteroGNN}(X_t, E_t)$
  Detect anomalous edges:
  for all $(u, v, k) \in E_t$ **do**
    $p_{\text{attack}} \leftarrow \text{Softmax}(\text{EdgeClassify}(Z_t(u), Z_t(v)))[1]$
    if $p_{\text{attack}} > \theta$ **then**
      Flag edge (u, v, k) as anomalous
    end if
  end for
end while

The proposed framework is designed to model and analyze network dynamics by integrating temporal and structural data derived from network traffic. The framework captures the evolution of a graph G_t over time, resulting in a sequence of graphs $\{G_t\}$ that reflects the temporal evolution of the network communications.

We adopt a *sliding window* strategy over the network flows, ensuring that each graph $G_t(V_t, E_t)$ reflects the most recent traffic within a chosen interval. At every update:

1. **Collect flows** from the last window size W (e.g., one hour) .
2. **Rebuild or update** the multi-edge graph G_t, adding nodes and edges as described below.
3. **Run community detection** (label propagation) to assign node labels.
4. **Use a heterogeneous GNN** to identify anomalies.

4.2.1. Constructing the time-evolving multi-edge graph

To capture changes in network communications, we construct a sequence of multi-edge graphs $\{G_t\}$, where each graph represents a time interval t.

Nodes: For each time t, V_t is the set of unique IP endpoints observed in the window $[t - W, t]$. Formally,

$$V_t = \{\text{SrcIP}, \text{DstIP} \mid \text{flows in } [t - W, t]\}. \tag{1}$$

Edges: E_t is a *multiset* of edges, allowing multiple distinct relationships (e.g., communication, context, knowledge) between the same pair of nodes (u, v). Each edge is labeled by its type k, reflecting the nature of the interaction:

$$\forall u, v \in V_t, \quad E_{uv,t} = \{\, e_k \mid e_k = (u, v, k) \wedge k \in \{\text{comm}, \text{context}, \text{knowledge}\}\}. \tag{2}$$

Hence, $E_t = \bigcup_{u,v} E_{uv,t}$.

At the beginning of each time interval, the multi-edge graph is constructed based on the current flow records in the buffer. The rolling window mechanism ensures that only flows within the window $[t-W, t]$ are used. Algorithm 2 outlines the steps to construct the time-evolving multi-edge graph.

Algorithm 2 Rolling Window Multi-Edge Graph Creation

Require: $\mathcal{B}_t$: buffer of flow records in $[t - W, t]$
Require: W: window size
Require: Δ_t: newly arrived flows in $[t - \Delta t, t]$
Ensure: $G_t(V_t, E_t)$: multi-edge graph for time t
Step 1: Update Buffer
Remove records in $\mathcal{B}_{t-\Delta t}$ older than $t - W$.
$\mathcal{B}_t \leftarrow \left(\mathcal{B}_{t-\Delta t} \setminus \{r \mid r.\text{time} < t - W\}\right) \cup \Delta_t$.
Step 2: Build Multi-Edge Graph
Initialize $V_t \leftarrow \emptyset$, $E_t \leftarrow \emptyset$
for all $r \in \mathcal{B}_t$ **do**
  $u \leftarrow r.\text{SrcIP}, \quad v \leftarrow r.\text{DstIP}$
  $V_t \leftarrow V_t \cup \{u, v\}$
  Insert Edge: $E_t \leftarrow E_t \cup \{(u, v, k)\}$, where k is based on flow characteristics:
  - k = comm if the flow represents Network communication.
  - k = context if the flow provides contextual data.
  - k = knowledge if the flow involves knowledge-based interactions.
end for
Define $G_t(V_t, E_t)$ as the multi-edge graph for time t.
return $G_t(V_t, E_t)$

4.2.2. Valuation function and edge types

We define an edge-weight valuation function $\Phi : E_t \to \mathbb{R}$ to compute weights for the edges in the multi-edge graph.

The edges are partitioned into three types:

$$E_{\text{comm}}, \quad E_{\text{context}}, \quad E_{\text{knowledge}}.$$

The valuation function assigns a weight $\Phi(u, v, k)$ based on the edge type k, as follows:

$$\Phi(u, v, k) = \begin{cases} \Phi^{(\text{comm})}(u, v, k), & (u, v, k) \in E_{\text{comm}}, \\ \Phi^{(\text{context})}(u, v, k), & (u, v, k) \in E_{\text{context}}, \\ \Phi^{(\text{know})}(u, v, k), & (u, v, k) \in E_{\text{knowledge}}. \end{cases} \tag{3}$$

Table 2
Comparison of community detection algorithms on TON-IoT dataset.

Algorithm	N. of communities detected	Processing time
Louvain	31	14.65 s
Infomap	72	3028.03 s
Label propagation	48	1.7 s

Each weight $\Phi^{(\cdot)}$ is computed as a weighted sum of normalized features:

$$\Phi^{(\mathrm{X})}(u, v, k) = \sum_{i=1}^{m_{\mathrm{X}}} \alpha_i^{\mathrm{X}} \cdot \mathrm{Norm}\left(x_i^{\mathrm{X}}(u, v, k)\right), \tag{4}$$

where:

- m_{X}: Number of features for edges of type X.
- $x_i^{\mathrm{X}}(u, v, k)$: The ith feature associated with edge (u, v, k) in E_{X}.
- $\mathrm{Norm}(\cdot)$: A normalization function (e.g., min–max).
- α_i^{X}: Learned coefficient for feature i of type X.

This flexible weighting mechanism ensures that each edge type contributes meaningfully to the analysis, reflecting the nature of interactions captured by the graph.

4.3. Community detection in the graph of interactions

After the graph of interactions has been constructed, we need to detect communities, i.e, nodes that form a cluster because they exhibit similar communication patterns. The community detection algorithm has to provide near real-time calculation. We discard spinglass [43] and walktrap [44] algorithms since they cannot handle disconnected graphs. We compare the computation time of the following community detection algorithms:

Louvain [45] applies a greedy approach to construct communities based on a modularity metric. This metric compares the density of edges inside vs. outside the community.

Infomap [46] explores the graph with a random walk: a region where the random walker stays longer as statistically expected is considered a community.

LPA [47] (Algo. 3) assigns each node a label, updating them based on neighboring labels to form communities with strong internal connections.

Table 2 illustrates the computation time of the different community detection algorithms with the TON-IoT dataset. The number of communities represents just an indicator since we do not have a ground truth in the original dataset. The dataset includes diverse IoT traffic, encompassing both benign and attack scenarios.

Infomap is very slow: we need more than 50 min to identify communities. Louvain is faster but we still need around 15 s, which is still computationally expensive. Only LPA can work in real-time, achieving close to 1 s latency.

LPA is selected due to its fast execution time, which is critical for our streaming-based anomaly detection pipeline. However, LPA comes with inherent limitations: it is non-deterministic, as it depends on random update orders and tie-breaking, and it may struggle to identify stable community structures in graphs with weakly defined clusters. We explicitly acknowledge these limitations in our design trade-off. While Louvain and Infomap also rely on greedy heuristics and suffer similar non-determinism, their significantly longer runtimes (Table 2) render them unsuitable for real-time settings.

In our case, LPA allows sub-second latency across rolling time windows while producing sufficient community separation for downstream anomaly scoring. We include the average number of detected communities and execution time over multiple runs to reflect variability, and note that future work may explore robust variants or hybrid schemes that balance community quality with latency constraints.

We propose to use the LPA technique in the remainder of the article, as it offers the best trade-off between community detection performance and execution speed. However, our approach is independent of the specific community detection algorithm employed. As such, others may opt for an algorithm with higher accuracy in complex graphs, albeit at the cost of increased computational demands.

4.3.1. Community detection via label propagation

For this task, we use LPA, which is well-suited for real-time applications and directed graphs.

After constructing G_t, we apply the Label Propagation Algorithm (LPA) to detect communities within the graph. In our use case, the input to LPA includes the multi-edge graph G_t and initial labels for each node, typically set as the node's unique identifier.

The LPA operates by iteratively updating node labels. During each iteration, a node adopts the label that is most frequent among its neighbors. For our directed multi-edge graphs, the directionality of edges is accounted for when determining neighbor relationships. The algorithm continues until the labels stabilize, meaning no further changes occur, and the communities have been identified.

Algorithm 3 provides the pseudocode for our adaptation of LPA to handle directed, multi-edge graphs efficiently.

Algorithm 3 Label Propagation Algorithm for Community Detection

Require: Graph $G_t(V_t, E_t)$; maximum iterations $M_{\max}$
Ensure: A labeling function $\pi : V_t \to C$ (community IDs)
Initialization:
for all $v \in V_t$ **do**
  $\pi(v) \leftarrow \mathrm{uniqueLabel}(v)$
end for
Repeat up to $M_{\max}$ times:
$changed \leftarrow false$
for all $v \in V_t$ (in random order) **do**
  Let $InNeighbors(v) = \{ u : (u \to v, k) \in E_t \}$ ▷ Collect inbound neighbors
  if $InNeighbors(v)$ not empty **then**
    $\ell \leftarrow$ most frequent label among $\{\pi(u) : u \in InNeighbors(v)\}$
    if $\ell \neq \pi(v)$ **then**
      $\pi(v) \leftarrow \ell$
      $changed \leftarrow true$
    end if
  end if
end for
if $changed = false$ **then**
  break
end if

The output of LPA is a set of community labels $\pi(v)$ for each node $v \in V_t$. Nodes sharing the same label are considered part of the same community. These labels are critical for the rest of the pipeline:

First, the community labels are added as node features in the graph, enriching the graph's structural and contextual information. This enrichment allows the HeteroGNN to better capture the relationships between nodes and edges. Second, the community structure provides insights into normal network behavior, highlighting anomalies as edges that deviate from the expected patterns. Finally, by incorporating these labels, the HeteroGNN can leverage community-level information to improve the classification of edges as benign or anomalous.

While prior literature often favors Louvain for its higher modularity scores in static graphs, our evaluation is driven by end-to-end system constraints rather than isolated community fidelity. The anomaly detection pipeline couples community detection with heterogeneous GNN-based classification, and its performance as reported in Table 2 is already high using LPA. At present, the community detector's role

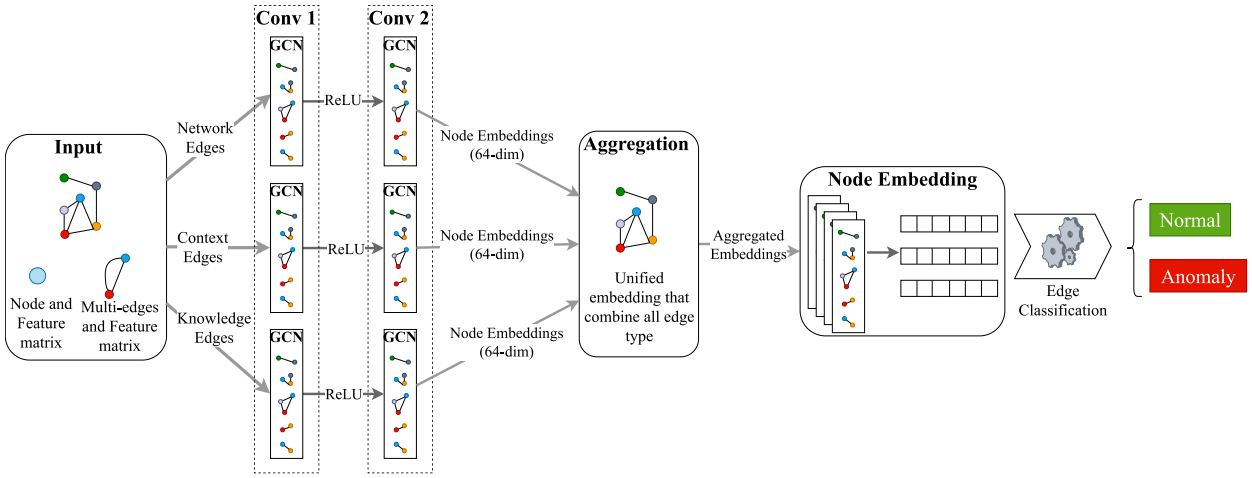


Fig. 3. Our proposed GNN model architecture.

is not to maximize modularity, but to extract discriminative subgraphs that allow efficient per-flow anomaly scoring.

It remains plausible that using Louvain might yield more granular community structures, potentially influencing detection boundaries. However, its processing time (14.65 seconds per window) would invalidate real-time performance, especially in high-volume IoT environments. Exploring its benefit in an offline or batch setting where higher latency is acceptable is a promising direction for future research, but beyond the scope of our streaming-focused architecture.

This step is a cornerstone of the pipeline, ensuring that the subsequent anomaly detection process benefits from a comprehensive understanding of both structural and behavioral patterns in the network.

4.4. Heterogeneous Graph Neural Network (HeteroGNN) for anomaly detection

The final stage of the pipeline focuses on anomaly detection using a HeteroGNN. The HeteroGNN takes as input the features of the network's hosts, represented as the vertices of G_t, and classifies each flow as either malicious or benign.

Specifically, the input features include the community label, in-degree, and out-degree of each host. The community label indicates the host's membership in a community, as determined in the previous step, helping to identify similar behaviors and efficiently detect anomalies.

In-degree represents the number of incoming connections to a node, serving as a measure of its popularity or importance, while out-degree reflects the number of outgoing connections, indicating the node's level of activity. For example, in an IoT network, a sensor node with an unusually high out-degree might signal anomalous behavior, such as data exfiltration or compromise. Furthermore, the dynamic nature of vertex degrees provides a temporal view of node interactions, which is crucial for detecting anomalies in time-evolving graphs.

The HeteroGNN employs multiple convolutional layers, each tailored to a specific edge type. For instance, separate `GCNConv` layers process the adjacency information for communication, context, and knowledge edges. Algorithm 4 provides a detailed description of the HeteroGNN architecture and its operation.

The graph G_t is processed by the HeteroGNN, which is specifically designed to classify edges (i.e, network flows) as benign or anomalous. The model employs type-specific convolutional layers, enabling it to independently process each edge type — communication, context, or knowledge — capturing their unique contributions. For instance, a communication edge might represent data exchange between devices, while a context edge could encode environmental conditions.

As depicted in Fig. 3, separate convolutional layers aggregate node features for each edge type, ensuring that distinct relationships are treated appropriately. The outputs of these convolutions are refined using batch normalization and ReLU activation functions, which stabilize the training process and introduce non-linearity. The refined outputs are then aggregated through summation, combining information from all edge types into unified node embeddings. These embeddings are further enhanced and dimensionally reduced through a multi-layer perceptron (MLP), increasing their expressiveness.

To classify edges, the model concatenates the embeddings of the source and destination nodes for each edge and passes them through an edge-specific classification head. This head outputs the likelihood of the edge being anomalous, enabling the identification of malicious interactions. The model is trained with a focal loss function to address the class imbalance inherent in anomaly detection tasks. This approach reduces the penalty for well-classified examples, focusing more on harder-to-classify anomalies, thus improving performance and robustness.

Algorithm 4 Heterogeneous GNN Training and Inference for Anomaly Detection

Require: $\mathcal{G}$: set of historic graphs, each $G_t(V_t, E_t)$ with node features X_t and edge labels Y_t
Require: Heterogeneous GNN model, anomaly threshold θ
Ensure: Anomaly edge set $\mathcal{A}$ over time
Training:
for $epoch \leftarrow 1$ to E_{max} **do**
  for all $(V_t, E_t, X_t, Y_t) \in \mathcal{G}$ **do**
    Node Embedding Generation:
    $Z_t \leftarrow \text{HeteroConv}(X_t, E_t)$
    for all edge types $(u, v, k) \in E_t$ with labels Y_t **do**
      $logits \leftarrow \text{EdgeClassify}(Z_t(u), Z_t(v))$
      $loss \leftarrow \text{FocalLoss}(logits, Y_t)$
      Backprop & update model weights
    end for
  end for
end for
Inference:
for all $G_t(V_t, E_t, X_t)$ in test set **do**
  Node Embedding Generation:
  $Z_t \leftarrow \text{HeteroConv}(X_t, E_t)$
  for all edge types $(u, v, k) \in E_t$ **do**
    $logits \leftarrow \text{EdgeClassify}(Z_t(u), Z_t(v))$
    $probs \leftarrow \text{Softmax}(logits)$
    $\mathcal{A}_t \leftarrow \{(u, v, k) \mid \text{probs}[1] > \theta\}$
  end for
end for
return all anomalies $\bigcup_t \mathcal{A}_t$

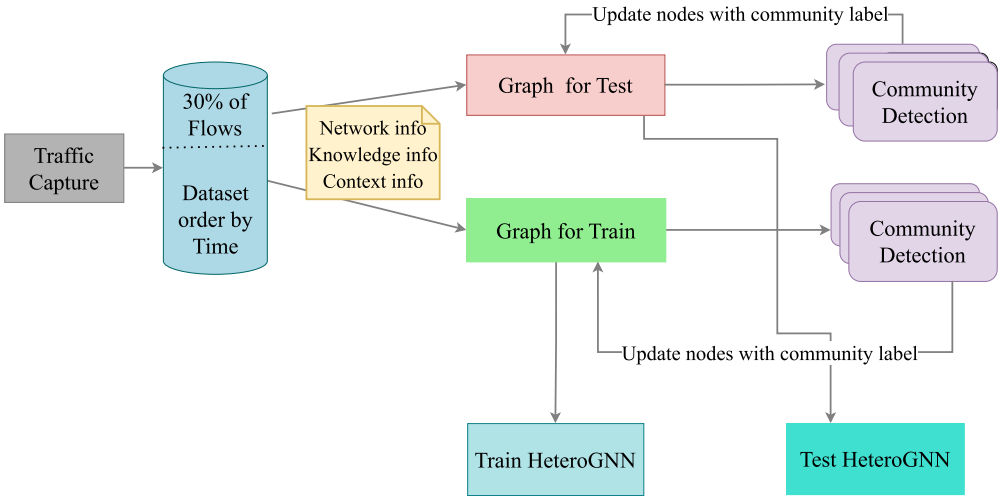


Fig. 4. Integrated framework for HeteroGNN anomaly detection.

The HeteroGNN's strength lies in its ability to integrate structural, contextual, and behavioral information. For example, in an industrial IoT setting, devices typically form tightly connected communities based on communication patterns. A device communicating outside its usual community or during unexpected time windows may signal an anomaly, which the model can flag for further investigation. Similarly, if a server interacts with a previously unassociated node, the disruption in community structure is detected as an anomalous edge.

5. Experimental evaluation

We first evaluate the performance of our approach in isolation, and then compare our solution with the state-of-the-art techniques with two datasets. This evaluation emphasizes both accuracy and real-time responsiveness. The model was trained using a mobile computing platform Apple M2 Pro with 10 CPU cores and 32 GB RAM. To ascertain the robustness of our approach, the model underwent training over 50 epochs, allowing us to extensively evaluate its learning capability and performance stability over time.

5.1. Method

We generate time-based snapshots of the network to capture temporal dynamics and construct multi-edge graphs with distinct edge types (e.g., network communication, context, knowledge). Each node is labeled based on the community it belongs to, facilitating anomaly detection. Fig. 4 illustrates our model, including all its components. This approach not only enhances the detection of network anomalies but also adapts to evolving traffic patterns, ensuring robust security in dynamic network environments.

5.2. Dataset preparation

Our performance evaluation relies on the popular CIC-ToN-IoT [48] and CIC-IDS2017 [49] datasets. These datasets are widely used in the literature as they regroup both benign traffic and simulated IoT specific attacks. They mimic a realistic network environment by including telemetry data of IoT services, network traffic, and operating system logs.

To clarify the scale and difficulty of the classification task, we explicitly report the anomaly distribution in each dataset.

CIC-IDS2017: This dataset comprises 2,830,743 bidirectional flow records generated using CIC-FlowMeter. Among these, 557,646 records (≈19.7%) are labeled as attacks. The remaining 2,273,097 records correspond to benign traffic. The attack flows span diverse categories, with the most prevalent being PortScan (≈158,000), DDoS (≈162,000), DoS-Hulk (≈128,000), and Brute-force (FTP + SSH, ≈13,000), followed by Botnet (≈1900), and a small number of Web and Infiltration attacks (fewer than 10,000 total).

CIC-ToN-IoT: We used the CIC-FlowMeter version of the dataset containing 1,846,373 bidirectional flow records. Of these, 461,934 records (25%) are labeled as attacks, while 1,384,439 are benign. The attack traffic includes DDoS (193,252), Scanning (105,699), Injection (72,534), Backdoor (19,126), DoS (19,243), Password cracking (15,277), MITM (1348), and Ransomware (149).

Stratified sampling was used to preserve class proportions during training and testing splits (70/30), ensuring consistent evaluation conditions.

We chose these datasets to showcase our approach's ability to handle large-scale networks and process extensive data volumes in dynamic environments, effectively addressing diverse and complex cyber threats. The CICIDS2017 dataset comprises 10,000 nodes and 2.8 million edges. TON-IoT, on the other hand, boasts 5000 nodes and approximately 19.4 million edges.

We acknowledge that the baseline methods used differ slightly between the CIC-IDS2017 and CIC-ToN-IoT evaluations. This decision was driven by compatibility constraints and implementation availability for certain models across specific dataset formats. For instance, DyEdgeGAT is designed for multivariate sensor time-series data and is not readily applicable to flow-based datasets such as CIC-IDS2017 without significant preprocessing or retraining. Despite this, we ensured that each dataset comparison included both graph-based and non-graph-based state-of-the-art models to support meaningful performance benchmarking.

5.3. Features selection

In this study, we use the Gini index to identify the most relevant features for anomaly detection. The Gini index is a criterion derived from decision trees in the CART (Classification and Regression Trees) algorithm and can handle both continuous and categorical variables. It quantifies a node's impurity in a decision tree (a lower Gini index denotes a more homogeneous node), and can therefore help to distinguish normal from abnormal traffic. We rank the features according to their importance for enhancing the model's capacity to recognize anomalies.

The Gini index for a set S containing classes C is formally defined as follows:

$$Gini(S) = 1 - \sum_{i=1}^{C} p_i^2 \tag{5}$$

where p_i is the proportion of samples in class i within the set S.

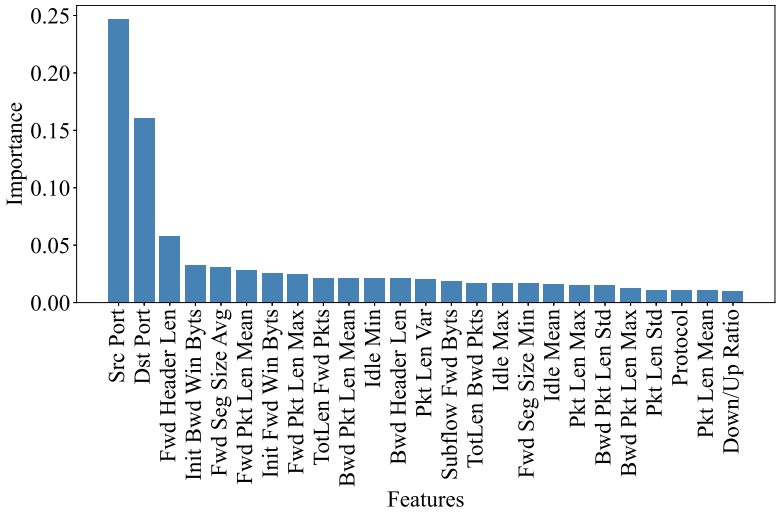


Fig. 5. Top 25 features importance based on their Gini Index.

5.3.1. Relevant features in the dataset

The results presented in Fig. 5 offer valuable insights into the key features for anomaly detection. Using the Gini Index, the analysis ranked feature importance from the CIC-ToN-IoT and CICIDS2017 datasets, identifying critical factors such as source and destination ports, forward packet length mean, and protocol-specific flags. These features highlight the significance of service-level attributes and communication patterns in distinguishing anomalous traffic. To enhance anomaly detection, the features were grouped into three categories for constructing multi-edge graphs:

1. Network Communication Features: Metrics like flow duration and packet counts represent communication intensity and patterns.
2. Contextual Features: Variables such as inter-arrival times and idle periods reveal behavioral and temporal anomalies.
3. Knowledge-Based Features: Attributes like packet size statistics and protocol flags encode logical relationships and expected norms.

This structured categorization facilitated the integration of feature importance into graph construction, improving the model's ability to detect diverse attack vectors across dynamic IoT networks.

5.4. Evaluation and metrics

Table 3 summarizes the architecture and training parameters used for the heterogeneous GNN model. These parameters are derived from the implementation, ensuring a balance between model complexity and computational efficiency. The configuration, including the use of Focal Loss and Adam optimizer, is specifically tailored to handle class imbalance and dynamic graph data, making the model robust for detecting anomalies in diverse IoT environments.

The model's performance is evaluated using the following classical metrics:

- **AUC**: Area Under the ROC Curve is formally defined as follows:

$$AUC = \int_0^1 \frac{TP}{TP+FN}\, d\left(\frac{FP}{TN+FP}\right) \tag{6}$$

  where TP, TN, FP, and FN denote the true positives, true negatives, false positives, and false negatives, respectively.
- **ROC Curve**: Plots True Positive Rate (TPR) against False Positive Rate (FPR), illustrating the model's performance across classification thresholds.

Table 3
Summary of GNN architecture and training parameters.

Parameter	Value
Hidden channels	64
Activation	ReLU
Learning rate	0.001
Weight decay	10^{-3}
Optimizer	Adam
Loss function	Focal Loss (with $\alpha = 0.25$, $\gamma = 2.0$)
Epochs	70

- **Precision, Recall, and F1-Score**: Evaluate the classification accuracy for anomalous edges.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{7}$$

$$Precision = \frac{TP}{TP+FP} \tag{8}$$

$$Recall = \frac{TP}{TP+FN} \tag{9}$$

$$F1-Score = \frac{2\times Precision\times Recall}{Precision+Recall} \tag{10}$$

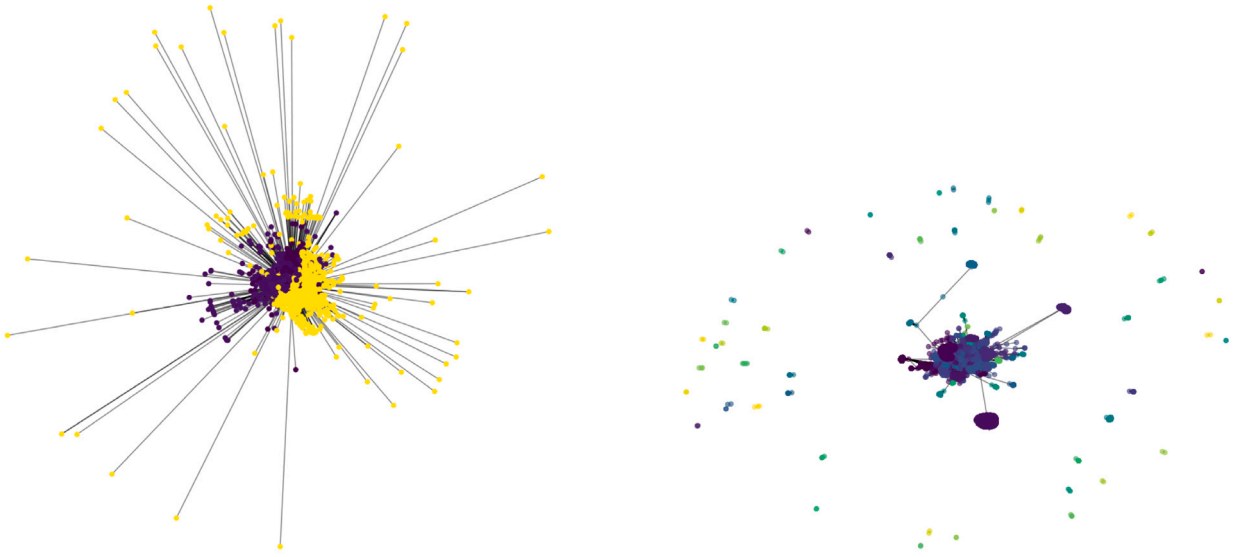
We conducted experiments on two widely used benchmark datasets to evaluate our proposed anomaly detection model: CIC-IDS2017 and CIC-ToN-IoT.

5.5. Communities

We first illustrate in Figs. 6(a) and 6(b) the communities identified in the two datasets used for the evaluation. In the CIC-IDS2017 dataset, the data is highly imbalanced, and the network lacks diversity in communication patterns (Fig. 6(a)). The network exhibits a centralized structure, characterized by a small number of highly connected hubs and limited interconnectivity among other nodes. Specifically, only five communities in the network contain more than three nodes, indicating a sparse and hierarchical communication pattern.

In contrast, the CIC-ToN-IoT dataset demonstrates a more complex and balanced structure. This dataset features 48 communities with three or more members, reflecting a broader and more distributed network topology (Fig. 6(b)). This network is characterized by more communities, higher interconnectivity among nodes, and less reliance on centralized hubs, resulting in a more decentralized and heterogeneous communication structure.

As it shows in Figs. 8(a) and 8(c), Wwith community detection included, the model has richer node features/embeddings (incorporating



(a) CIC-IDS2017 (b) CIC-ToN-IoT

Fig. 6. Network graph of communities.

Table 4
Comparison of edge configurations for anomaly detection.

Edge configuration	Precision	Recall	F1-Score	Accuracy	Area under curve
CICIDS2017 2 edges	0.8996	1.0000	0.9472	0.9442	0.9973
CICIDS2017 3 edges	0.9972	1.0000	0.9986	0.9986	0.9973
CIC-ToN-IoT 2 edges	0.9778	1.0000	0.9888	0.9965	1.0000
CIC-ToN-IoT 3 edges	0.9888	1.0000	0.9944	0.9982	1.0000
ToN without community 3 edges	0.9615	0.8523	0.9036	0.9719	0.9922

community labels or structure), which helps it discriminate benign from attack edges more accurately. As shown in Fig. 8(a) confusion matrix, there are fewer false positives and false negatives compared to the run without community detection. This leads to higher precision and recall overall. By contrast, removing community information deprives the model of helpful structural context, slightly reducing its ability to correctly classify attacks.

5.6. Impact of the number of contexts

We first investigate the relevance of our context-aware design. Table 4 illustrates the efficiency of the method to detect anomalies when considering only 2 types of context (`Network Communication`, `knowledge`) vs. 3 types of context (`Network Communication`, `context`, `knowledge`).

In the CICIDS2017 dataset, precision increases from 89.96% in the 2-edge configuration to 99.72% in the 3-edge configuration, while the F1-score improves significantly from 94.72% to 99.86%. Similarly, for the CIC-ToN-IoT dataset, the precision improves from 97.78% to 98.88%, with the F1-score increasing from 98.88% to 99.44%. Importantly, recall remains consistently at 100% across all configurations, indicating the model's robustness in detecting true anomalies.

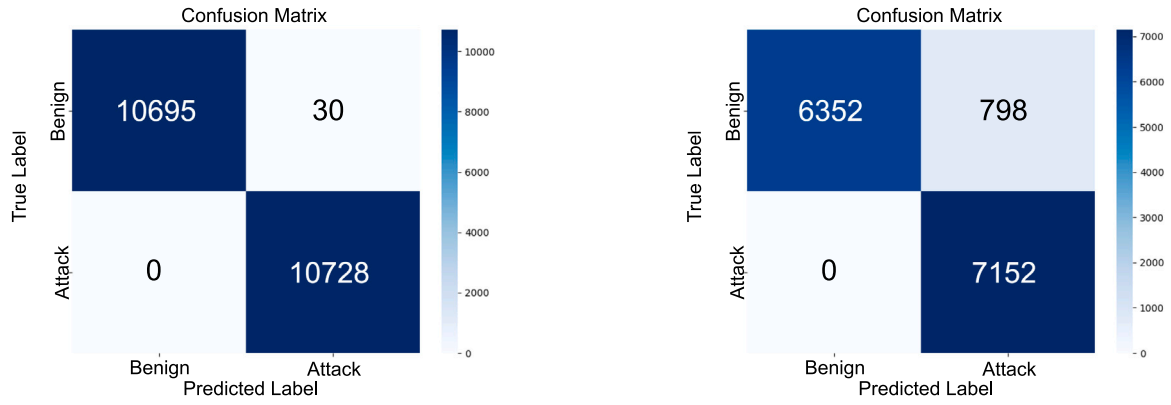
We can conclude that incorporating all contexts together enhances **Precision**, **F1-score**, and **Accuracy**. This demonstrates the agility of our architecture in accommodating multiple contexts and highlights the importance of context-aware decision-making in reducing false positives. Notably, this improvement in anomaly detection capability is achieved without compromising recall.

To further analyze these outcomes, Figs. 7(a) and 7(b) present the corresponding confusion matrices. The 3-edge model exhibits fewer false positives and false negatives, illustrating its robust classification of normal versus attack traffic. While the 2-edge model remains competent, its slightly higher number of misclassifications underscores the benefit of leveraging a richer, more interconnected representation in the 3-edge scenario. Hence, for this more heterogeneous dataset, the additional edge information clearly enhances the model's ability to detect anomalies accurately.

Both variants again perform well, with the 3-edge model achieving an F1 score of 0.9944 and the 2-edge model attaining an F1 score of 0.9888. Although the improvement of the 3-edge approach is not as dramatic as in the CIC-IDS2017 case, this narrower gap suggests that IoT traffic patterns in the CIC-ToN-IoT dataset are relatively straightforward to model. Still, capturing additional structural relationships through a third edge slightly strengthens the model's discrimination capability.

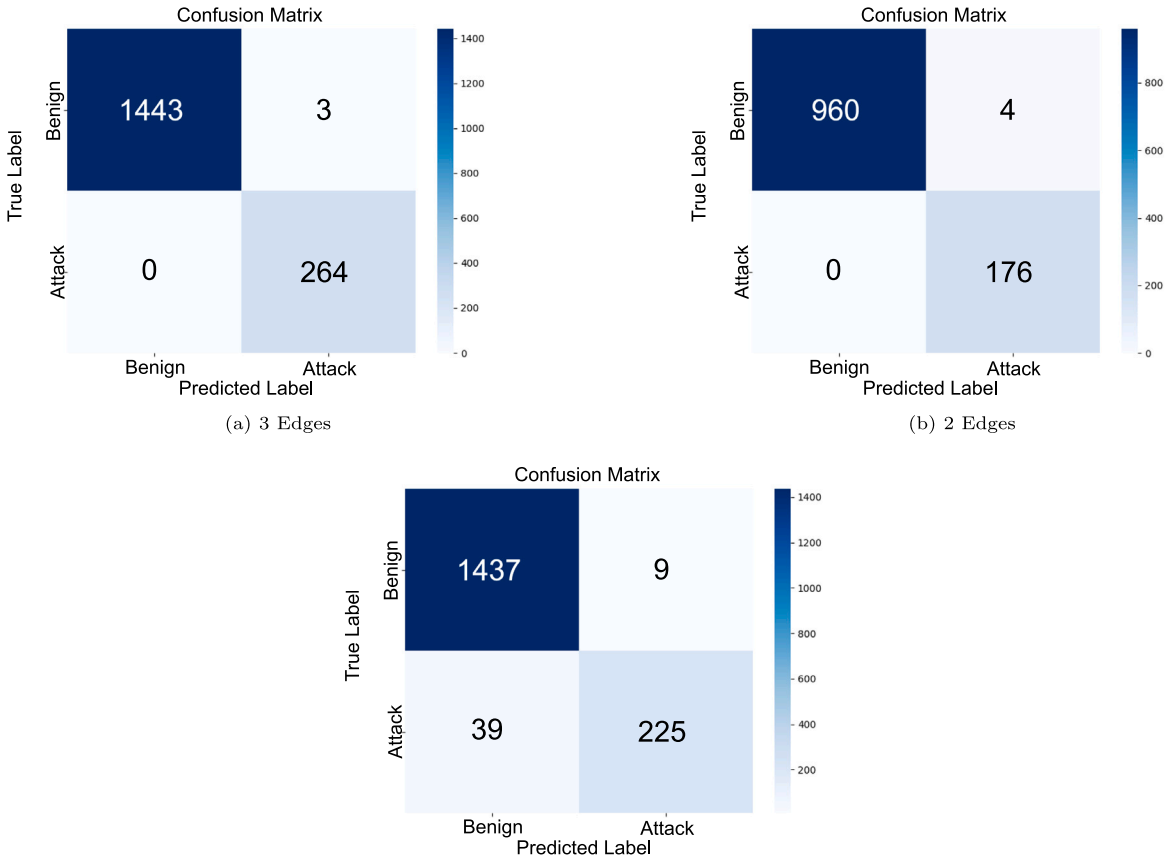
The confusion matrices in Figs. 8(a) and 8(b) confirm these observations, revealing that the 3-edge approach yields marginally fewer misclassifications than the 2-edge approach. This translates to more consistent detection of intrusions with minimal false alerts. Despite the reduced margin of difference, the results demonstrate that expanding the network graph representation from two edges to three edges can still confer measurable advantages in capturing subtle anomalies inherent to IoT systems.

Thus, our findings reinforce that the proposed 3-edge model outperforms the 2-edge version across both benchmarks. The performance benefit is more pronounced in the CIC-IDS2017 dataset, which features a broader range of attack types and traffic patterns, thereby amplifying the value of additional structural information in the model. In contrast, the CIC-ToN-IoT dataset presents a narrower behavioral spectrum, but still benefits from more extensive graph connections. These results validate our hypothesis that incorporating a richer graph structure enables improved anomaly detection, particularly in complex network environments.



(a) 3 Edges (b) 2 Edges

Fig. 7. Comparative confusion matrix for CIC-IDS2017 dataset.



(a) 3 Edges (b) 2 Edges

(c) 3 Edges Without Considering Community detection

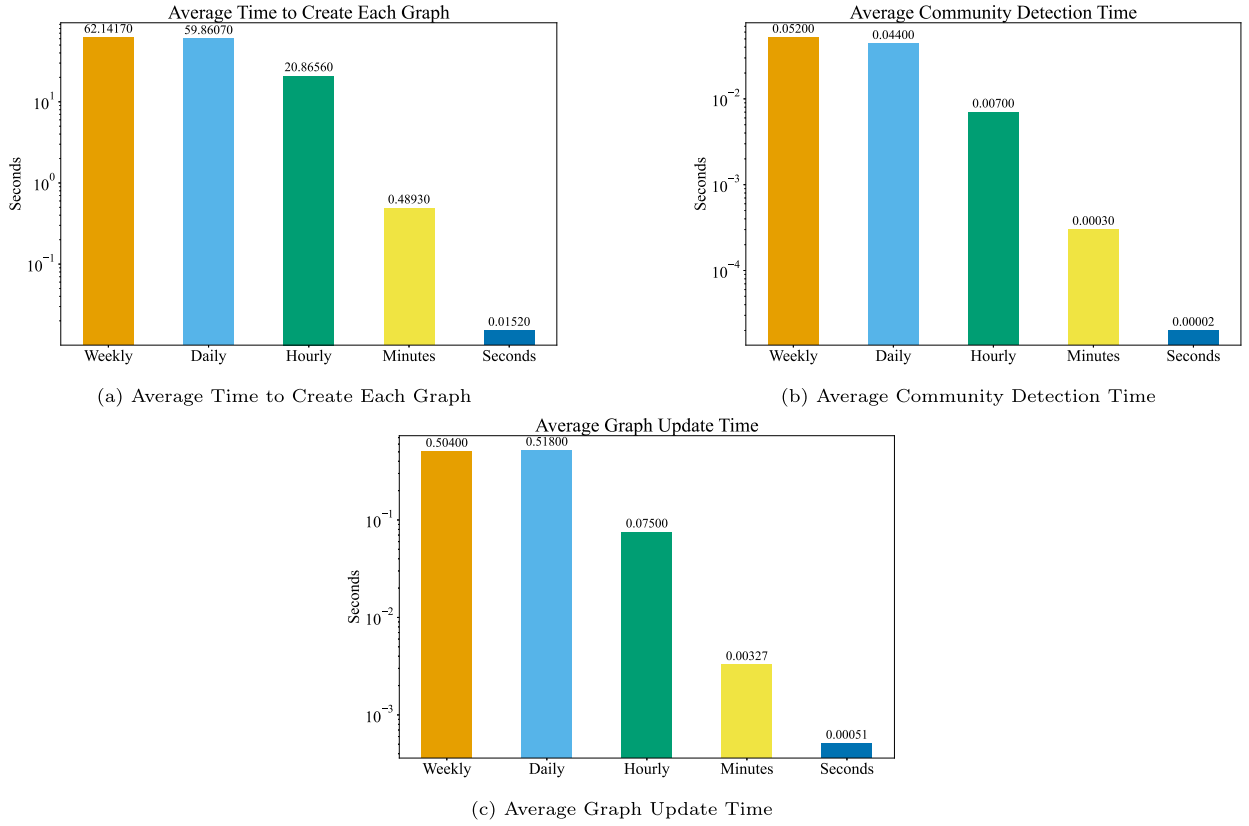
Fig. 8. Comparative confusion matrix for CIC-ToN-IoT dataset.

5.7. Impact of the time scale

To evaluate how different historical window sizes affect both system performance and anomaly detection reliability, we measured graph creation, community detection, and update times across five time scales: weekly, daily, hourly, minutes, and seconds (Fig. 9). The results reveal a clear trade-off between computational cost and temporal granularity.

As expected, using smaller windows significantly reduces the processing time per operation. For example, average graph creation time decreases from 62.14 s at the weekly scale to only 0.0152 s for secondly windows. Similar improvements are observed in community detection time (from 0.0520 to 0.00002 s) and graph update time (from 0.5040 to 0.00051 s). These measurements confirm the system's capacity for near-real-time operation, with GNN inference itself requiring only 0.0075 seconds per flow.

However, time scale also influences graph density and detection accuracy. Second-level windows often generate sparse graphs with limited context, which can reduce detection reliability. Conversely, large windows (e.g., daily or weekly) result in high-density graphs that are costly to maintain and slow to process. During empirical



(a) Average Time to Create Each Graph

(b) Average Community Detection Time

(c) Average Graph Update Time

Fig. 9. Performance trends across different time scales.

testing, we found that *minute-level* windows provided the best balance capturing sufficient temporal context for accurate anomaly detection while keeping update and inference latency within acceptable bounds.

This trade-off suggests that minute-level aggregation is optimal for real-world deployments that require low-latency detection without compromising detection fidelity or exhausting computational resources.

5.7.1. End-to-end processing time

Beyond per-component runtimes, we measure the total time per sliding-window update at our **minute-level setting** ($W = 60\,\text{s}$, $\Delta t = 30\,\text{s}$). Summing from Fig. 9:

$$0.4893\ \text{s (graph creation)} + 0.0003\ \text{s (community detection)} + 0.00327\ \text{s (graph update)} + 0.0075\ \text{s (GNN inference)} \approx 0.5004\ \text{s/update}.$$

This yields approximately 2 updates/s, satisfying real-time requirements.

5.8. Comparison with state-of-the-art techniques

We compare here our proposition with state-of-the-art techniques. This study's comparison is based on methods for finding anomalies that were tested on the CIC-TON-IoT and CIC-IDS2017 datasets. For the implementation, we used a server containing an Intel Xeon Gold 6330 @ 2.00 GHz with an RTX 3090 GPU and 24 GB of memory.

5.8.1. CIC-ToN-IoT dataset

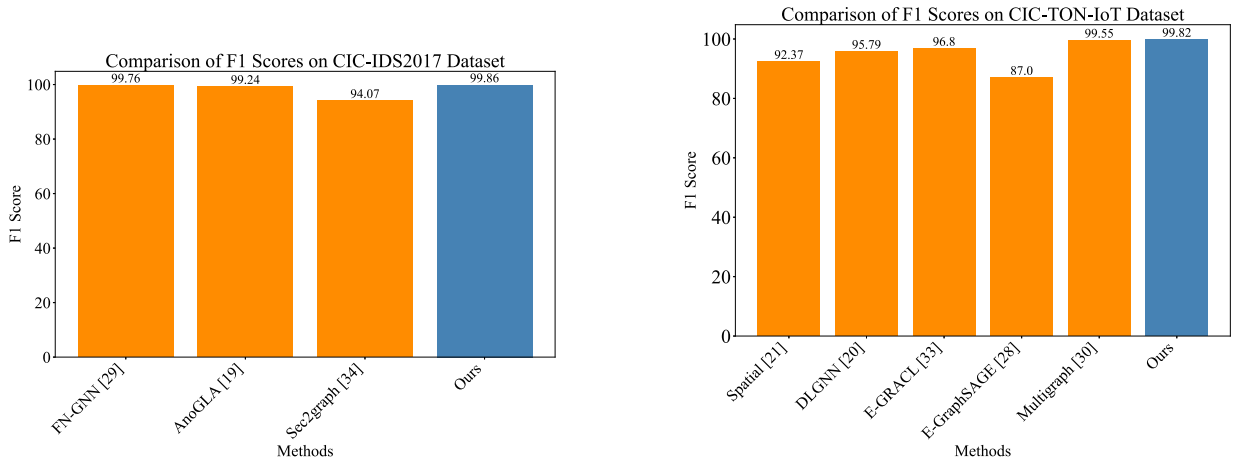
Fig. 10(b) contrasts the F1 scores of multiple anomaly detection approaches applied to the CIC-ToN-IoT dataset. We incorporate results for Spatial [50], DLGNN [51], E-GRACLN [28], E-GraphSAGE [27], and Multigraph [29]. Notably, E-GraphSAGE attains a commendable F1 score of 96.8%, highlighting the value of graph-based message passing in capturing relationships among IoT nodes. Meanwhile, DL-GNN (95.79%) and Spatial (92.37%) demonstrate robust but slightly lower performance, suggesting that while these architectures handle IoT-related features effectively, they may miss certain deeper relational or contextual cues. Multigraph achieves a particularly high F1 score of 99.55%, indicating that additional layers of graph abstraction can significantly boost detection capabilities in IoT settings.

5.8.2. CIC-IDS2017 dataset

Fig. 10(a) illustrates the F1 performance on the CIC-IDS2017 dataset for our approach in comparison with FN-GNN [52], AnoGLA [53], and Sec2graph [26]. Both FN-GNN and AnoGLA surpass 99% F1, reflecting their sophisticated graph-based learning and attention mechanisms tailored for cybersecurity data. By contrast, Sec2graph achieves 94.07%, which is competitive but indicates potential limitations in capturing a broader range of threat behaviors within enterprise traffic. Our approach reaches an F1 score of 99.86%, improving slightly over FN-GNN and AnoGLA, and markedly exceeding Sec2graph.

While CIC-ToN-IoT traffic often displays more homogeneous, "community-like" structures—where device behaviors and protocols follow narrower patterns—the CIC-IDS2017 dataset encompasses diverse enterprise traffic spanning multiple services, protocols, and user behaviors. Such variability can make classification more challenging, as malicious flows might resemble legitimate but less common activities. Nonetheless, our context-aware and community-enhanced graph construction proves equally effective here. By identifying cohesive sub-networks and analyzing node-level anomalies within these communities, our method achieves robust detection despite the dataset's more heterogeneous nature.

In Fig. 10(a), the IoT-based dataset (CIC-ToN-IoT) shows more consistent, community-like structures (see Fig. 6) than the diverse enterprise-style traffic in Fig. 10(b) (CIC-IDS2017). This tighter distribution of "normal" IoT traffic makes anomalies stand out more



(a) CIC-IDS2017 (b) CIC-ToN-IoT

Fig. 10. Comparison of different methods.

distinctly, leading to better classification performance. By contrast, enterprise traffic encompasses various protocols and behaviors, causing more overlap between benign and malicious samples. Additionally, the flow-based time granularity in both figures typically on the order of 1–5 s, ensures that each node or point represents an aggregated snapshot of network activity, further emphasizing the clear structural patterns in the IoT dataset.

Finally, the high F1 scores observed underscore how effectively our approach balances precision and recall. Particularly in security contexts, an accurate detection system must not only identify threats but do so with minimal false alarms. Excessive false positives can overwhelm analysts, while false negatives allow intrusions to remain undetected. That our model consistently reports near-ideal F1 scores (above 99%) in two very different environments speaks to its versatility and potential readiness for production environments. These outcomes lay a foundation for future work aimed at extending the framework to more granular real-time intrusion detection, cross-dataset transfer learning, or distributed implementations that further reduce the computational overhead on any single node.

6. Conclusions and discussion

In this work, we present an anomaly detection framework tailored for complex network environments, leveraging a HeteroGNN and community detection to analyze dynamic network interactions. The framework constructs multi-edge graphs to represent diverse relationships (network communication, context, knowledge) and dynamically captures the temporal evolution of network behavior through traffic analysis. By training the model on labeled data, our framework effectively identifies deviations from normal network behavior. This approach provides accurate detection of anomalous interactions in resource-constrained IoT environments. Unlike traditional techniques, which often require a trade-off between extensive training and computational complexity, our method is computationally efficient and well-suited for real-time applications.

Future work will focus on integrating unsupervised techniques — such as autoencoders — into the current framework. By combining the supervised HeteroGNN approach with an autoencoder, the system could more effectively detect previously unseen attacks through the identification of deviations from established behavioral baselines. This hybrid strategy would not only broaden the framework's applicability to evolving and unforeseen threats but also preserve its computational efficiency, making it suitable for real-time deployment.

In addition, we plan to extend the pipeline by integrating the Louvain algorithm for community detection and evaluate its impact on downstream anomaly detection accuracy. We plan to investigate the trade-off between improved anomaly detection accuracy and the increased computational overhead required to achieve it.

CRediT authorship contribution statement

Fatemeh Stodt: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology. **Christoph Reich:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition. **Fabrice Theoleyre:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Christoph Reich and Fatemeh Stodt reports financial support was provided by Austrian Federal Ministry of Education Science and Research. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by the Federal Ministry of Education and Research (BMBF), Germany under reference number COSMIC-X 02J21D144, and supervised by Projektträger Karlsruhe (PTKA).

Data availability

We provide the implementation of our code in GitHub (ref [8] in the paper)

References

[1] F. Ehsani-Besheli, H.R. Zarandi, Context-aware anomaly detection in embedded systems, in: DepCoS-RELCOMEX, Springer, 2018, pp. 151–165.
[2] J.M. Estevez-Tapiador, P. Garcia-Teodoro, J.E. Diaz-Verdejo, Anomaly detection methods in wired networks: a survey and taxonomy, Comput. Commun. 27 (16) (2004) 1569–1584.
[3] H.-J. Liao, C.-H.R. Lin, Y.-C. Lin, K.-Y. Tung, Intrusion detection system: A comprehensive review, J. Netw. Comput. Appl. 36 (1) (2013) 16–24.
[4] L. Wang, M. Han, X. Li, N. Zhang, H. Cheng, Review of classification methods on unbalanced data sets, IEEE Access 9 (2021) 64606–64628.

[5] R. Yasaei, M.A.A. Faruque, Context-aware adaptive anomaly detection in IoT systems, in: Embedded Machine Learning for Cyber-Physical, IoT, and Edge Computing: Use Cases and Emerging Challenges, Springer, 2023, pp. 177–200.

[6] G. Dong, M. Tang, Z. Wang, J. Gao, S. Guo, L. Cai, R. Gutierrez, B. Campbel, L.E. Barnes, M. Boukhechba, Graph neural networks in IoT: A survey, ACM Trans. Sens. Networks 19 (2) (2023) 1–50.

[7] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, M. Sun, Graph neural networks: A review of methods and applications, AI Open 1 (2020) 57–81.

[8] F. Stodt, Graph, 2025, GitHub repository, URL https://github.com/f11691/GraphIDS. [Online; (Accessed 20 January 2025)].

[9] F. Li, A. Shinde, Y. Shi, J. Ye, X.-Y. Li, W. Song, System statistics learning-based IoT security: Feasibility and suitability, IEEE Internet Things J. 6 (4) (2019) 6396–6403.

[10] B. Schölkopf, R.C. Williamson, A. Smola, J. Shawe-Taylor, J. Platt, Support vector method for novelty detection, Adv. Neural Inf. Process. Syst. 12 (1999).

[11] I. Fosić, D. Žagar, K. Grgić, V. Križanović, Anomaly detection in NetFlow network traffic using supervised machine learning algorithms, J. Ind. Inf. Integr. 33 (2023) 100466, http://dx.doi.org/10.1016/j.jii.2023.100466.

[12] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, H. Wu, Towards early and accurate network intrusion detection using graph embedding, IEEE Trans. Inf. Forensics Secur. (2023).

[13] S. Zavrak, M. Iskefiyeli, Anomaly-based intrusion detection from network flow features using variational autoencoder, IEEE Access 8 (2020) 108346–108358.

[14] B. Min, J. Yoo, S. Kim, D. Shin, D. Shin, Network anomaly detection using memory-augmented deep autoencoder, IEEE Access 9 (2021) 104695–104706.

[15] I. Ullah, Q.H. Mahmoud, Design and development of RNN anomaly detection model for IoT networks, IEEE Access 10 (2022) 62722–62750, http://dx.doi.org/10.1109/ACCESS.2022.3176317.

[16] H.D. Trinh, L. Giupponi, P. Dini, Urban anomaly detection by processing mobile traffic traces with LSTM neural networks, in: 2019 16th Annual IEEE International Conference on Sensing, Communication, and Networking, SECON, 2019, pp. 1–8, http://dx.doi.org/10.1109/SAHCN.2019.8824981.

[17] M. Said Elsayed, N.-A. Le-Khac, S. Dev, A.D. Jurcut, Network anomaly detection using LSTM based autoencoder, in: Symposium on QoS and Security for Wireless and Mobile Networks, Q2SWinet, ACM, 2020, pp. 37–45, http://dx.doi.org/10.1145/3416013.3426457.

[18] L. Mhamdi, D. McLernon, F. El-Moussa, S.A.R. Zaidi, M. Ghogho, T. Tang, A deep learning approach combining autoencoder with one-class SVM for DDoS attack detection in SDNs, in: 2020 IEEE Eighth International Conference on Communications and Networking, ComNet, IEEE, 2020, pp. 1–6.

[19] A.K. Sahu, S. Sharma, M. Tanveer, R. Raja, Internet of things attack detection using hybrid deep learning model, Comput. Commun. 176 (2021) 146–154, http://dx.doi.org/10.1016/j.comcom.2021.05.024.

[20] C. Qiu, Y. Geng, J. Lu, K. Chen, S. Zhu, Y. Su, G. Nan, C. Zhang, J. Fu, Q. Cui, et al., 3D-IDS: Doubly disentangled dynamic intrusion detection, in: ACM SIGKDD, 2023, pp. 1965–1977.

[21] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martinez-del Rincon, D. Siracusa, LUCID: A practical, lightweight deep learning solution for DDoS attack detection, IEEE Trans. Netw. Serv. Manag. 17 (2) (2020) 876–889.

[22] C. Berge, The Theory of Graphs, Wiley, 1962.

[23] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive survey on graph neural networks, IEEE Trans. Neural Networks Learn. Syst. 32 (1) (2021) 4–24, http://dx.doi.org/10.1109/TNNLS.2020.2978386.

[24] X. Zhou, W. Liang, W. Li, K. Yan, S. Shimizu, K.I.-K. Wang, Hierarchical adversarial attacks against graph-neural-network-based IoT network intrusion detection system, IEEE Internet Things J. 9 (12) (2022) 9310–9319, http://dx.doi.org/10.1109/JIOT.2021.3130434.

[25] Y. Wu, H.-N. Dai, H. Tang, Graph neural networks for anomaly detection in industrial internet of things, IEEE Internet Things J. 9 (12) (2021) 9214–9231.

[26] L. Leichtnam, E. Totel, N. Prigent, L. Mé, Sec2graph: Network attack detection based on novelty detection on graph structured data, in: International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, DIMVA, Springer, Lisbon, Portugal, 2020, pp. 238–258.

[27] W.W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, M. Portmann, E-graphsage: A graph neural network based intrusion detection system for iot, in: NOMS, IEEE/IFIP, 2022, pp. 1–9, http://dx.doi.org/10.1109/NOMS54207.2022.9789878.

[28] L. Lin, Q. Zhong, J. Qiu, Z. Liang, E-GRACL: an IoT intrusion detection system based on graph neural networks, J. Supercomput. 81 (1) (2025) 42.

[29] T. Altaf, X. Wang, W. Ni, G. Yu, R.P. Liu, R. Braun, A new concatenated multigraph neural network for IoT intrusion detection, Internet Things 22 (2023) 100818.

[30] H. Friji, A. Olivereau, M. Sarkiss, Efficient network representation for GNN-based intrusion detection, in: ACNS, Springer, 2023, pp. 532–554.

[31] C. Park, D. Kim, J. Han, H. Yu, Unsupervised attributed multiplex network embedding, in: AAAI, Vol. 34, 2020, pp. 5371–5378.

[32] M. Yoon, B. Hooi, K. Shin, C. Faloutsos, Fast and accurate anomaly detection in dynamic graphs with a two-pronged approach, in: ACM SIGKDD, 2019, pp. 647–657.

[33] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, M. Bronstein, Temporal graph networks for deep learning on dynamic graphs, in: ICML Workshop on Graph Representation Learning, 2020, pp. 1–9.

[34] I.J. King, H.H. Huang, Euler: Detecting network lateral movement via scalable temporal link prediction, ACM Trans. Priv. Secur. 26 (3) (2023) 1–36.

[35] A. Mitra, P. Vijayan, R. Sanasam, D. Goswami, S. Parthasarathy, B. Ravindran, Semi-supervised deep learning for multiplex networks, in: ACM SIGKDD, 2021, pp. 1234–1244.

[36] P. Kowalski, A.-L. Jousselme, Context-awareness for information correction and reasoning in evidence theory, Internat. J. Approx. Reason. 153 (2023) 29–48.

[37] A. Rullo, D. Midi, A. Mudjerikar, E. Bertino, Kalis2.0—A secaas-based context-aware self-adaptive intrusion detection system for IoT, IEEE Internet Things J. 11 (7) (2024) 12579–12601, http://dx.doi.org/10.1109/JIOT.2023.3333948.

[38] R. Yasaei, F. Hernandez, M.A. Al Faruque, IoT-CAD: Context-aware adaptive anomaly detection in IoT systems through sensor association, in: 2020 IEEE/ACM International Conference on Computer Aided Design, ICCAD, IEEE, 2020, pp. 1–9.

[39] H.K. Kalutarage, M.O. Al-Kadri, M. Cheah, G. Madzudzo, Context-aware anomaly detector for monitoring cyber attacks on automotive CAN bus, in: ACM Computer Science in Cars Symposium, 2019, pp. 1–8, http://dx.doi.org/10.1145/3359999.3360496.

[40] M. Zhao, O. Fink, Dyedgegat: Dynamic edge via graph attention for early fault detection in IIoT systems, IEEE Internet Things J. 11 (13) (2024) 22950–22965.

[41] S.H. Mekala, Z. Baig, A. Anwar, S. Zeadally, Cybersecurity for industrial IoT (IIoT): Threats, countermeasures, challenges and future directions, Comput. Commun. 208 (2023) 294–320, http://dx.doi.org/10.1016/j.comcom.2023.06.020.

[42] C. Zhang, D. Song, C. Huang, A. Swami, N.V. Chawla, Heterogeneous graph neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 793–803.

[43] J. Reichardt, S. Bornholdt, Statistical mechanics of community detection, Phys. Rev. E—Stat. Nonlinear Soft Matter Phys. 74 (1) (2006) 016110.

[44] P. Pons, M. Latapy, Computing communities in large networks using random walks, in: Computer and Information Sciences-ISCIS 2005: 20th International Symposium, Istanbul, Turkey, October 26-28, 2005. Proceedings 20, Springer, 2005, pp. 284–293.

[45] S. Ghosh, M. Halappanavar, A. Tumeo, A. Kalyanaraman, H. Lu, D. Chavarria-Miranda, A. Khan, A. Gebremedhin, Distributed Louvain algorithm for graph community detection, in: IPDPS, IEEE, 2018, pp. 885–895.

[46] M. Rosvall, D. Axelsson, C.T. Bergstrom, The map equation, Eur. Phys. J. Special Top. 178 (1) (2009) 13–23.

[47] U.N. Raghavan, R. Albert, S. Kumara, Near linear time algorithm to detect community structures in large-scale networks, Phys. Rev. E—Stat. Nonlinear Soft Matter Phys. 76 (3) (2007) 036106.

[48] M. Sarhan, S. Layeghy, M. Portmann, Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection, Big Data Res. 30 (2022) 100359.

[49] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, et al., Toward generating a new intrusion detection dataset and intrusion traffic characterization, ICISSp 1 (2018) 108–116.

[50] X. Wang, X. Wang, M. He, M. Zhang, Z. Lu, Spatial-temporal graph model based on attention mechanism for anomalous IoT intrusion detection, IEEE Trans. Ind. Informatics (2023).

[51] G. Duan, H. Lv, H. Wang, G. Feng, Application of a dynamic line graph neural network for intrusion detection with semisupervised learning, IEEE Trans. Inf. Forensics Secur. 18 (2022) 699–714.

[52] D.-H. Tran, M. Park, FN-GNN: A novel graph embedding approach for enhancing graph neural networks in network intrusion detection systems, Appl. Sci. 14 (16) (2024) 6932.

[53] Q. Ding, J. Li, AnoGLA: An efficient scheme to improve network anomaly detection, J. Inf. Secur. Appl. 66 (2022) 103149.